UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Docente: Marlon Fuentes

HOJA DE TRABAJO 3

CUDA Y PARALELISMO DE DATOS

Suma vectorial, clasificación por umbral y comparación de rendimiento CPU/GPU

Estudiante Pablo Daniel Barillas Moreno
Carné 22193
Sección 40
Semestre 2, 2026

Índice

1. Trazabilidad con el enunciado 1

2. Fundamentos del modelo CUDA 2
2.1. Flujo general . . . 2

3. Parte 1: suma de arreglos en GPU 2
3.1. Objetivo y datos . . . 2
3.2. Kernel . . . 2
3.3. Resultado observado . . . 3
3.4. Respuestas requeridas . . . 3
3.4.1. ¿Qué partes se ejecutan en CPU y cuáles en GPU? . . . 4
3.4.2. ¿Qué ocurre al cambiar grid y block? . . . 4
3.4.3. ¿Qué ocurre si no se reserva o libera memoria? . . . 4
3.4.4. ¿Qué ocurre si no se transfiere el resultado a CPU? . . . 4

4. Parte 2: clasificación por umbral 4
4.1. Relación con la plantilla . . . 4
4.2. Regla de clasificación . . . 4
4.3. Resultados . . . 5

5. Parte 3: versión CPU y comparación de tiempos 6
5.1. Versión exclusivamente CPU . . . 6
5.2. Diseño de la comparación justa . . . 7
5.3. Resultados medidos . . . 8

6. Validación integral 8

7. Reproducibilidad y limitaciones 9
7.1. Reproducibilidad . . . 9
7.2. Factores que afectan el tiempo . . . 9

8. Conclusiones 9

A. Archivos principales 9

Resumen ejecutivo

Esta hoja de trabajo desarrolla tres competencias fundamentales de CUDA. La primera parte implementa una suma de arreglos en GPU y analiza la separación entre host y device. La segunda completa la plantilla `vacio.cu` para clasificar valores respecto de un umbral. La tercera construye una versión secuencial en CPU y un comparador reproducible CPU/GPU que distingue el tiempo del kernel del costo completo de transferencias. Las tres partes fueron ejecutadas y validadas en una NVIDIA GeForce RTX 4050 Laptop GPU. El kernel alcanzó una aceleración de 7.72x respecto de CPU; al incluir transferencias, la aceleración extremo a extremo fue 0.45x. El resultado demuestra que una mejora en cómputo no implica necesariamente una mejora en el programa completo.

1. Trazabilidad con el enunciado

El enunciado oficial se conserva dentro del repositorio, junto con la plantilla original de la Parte 2. La Tabla 1 relaciona cada requisito con su evidencia.

Requisito	Implementación	Estado
Suma de A y B en GPU	`source/parte1_suma_cuda.cu`: memoria, copias, kernel e impresión	**Cumplido**
Respuestas de Parte 1	Sección 3.4 de este informe y README	**Cumplido**
Completar `vacio.cu`	`source/vacio.cu`, contrastado con la plantilla original	**Cumplido**
Versión exclusivamente CPU	`source/parte3_cpu_vs_gpu.cpp`	**Cumplido**
Medición CPU y GPU	`source/parte3_comparacion_cpu_gpu.cu`, con eventos CUDA y reloj CPU	**Cumplido**
Informe en PDF	Fuente LaTeX, capturas integradas y salida en `output/pdf`	**Cumplido**
No incluir ejecutables	`build/` excluido mediante `.gitignore`	**Cumplido**

Cuadro 1: Trazabilidad de requisitos y entregables.

Entorno de ejecución validado

La medición final se realizó en WSL con CUDA 12.4 y una NVIDIA GeForce RTX 4050 Laptop GPU. El controlador reportó la versión 621.01 y 6,141 MiB de memoria. Los compiladores `g++` y `nvcc`, además de `nvidia-smi`, fueron verificados antes de ejecutar la suite.

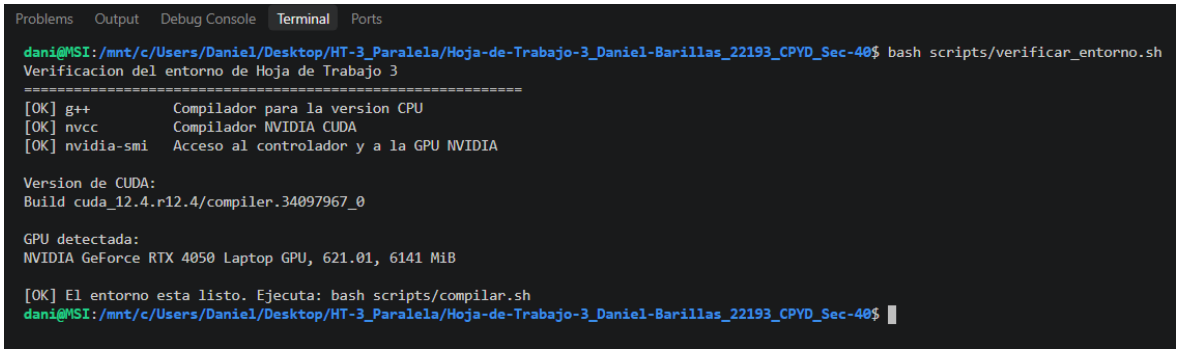


Figura 1: Verificación del entorno WSL, CUDA y GPU antes de compilar.

2. Fundamentos del modelo CUDA

Un programa CUDA combina dos espacios de ejecución. El *host*, que corresponde a la CPU, coordina el programa y administra recursos. El *device*, que corresponde a la GPU, ejecuta kernels sobre una gran cantidad de hilos. En esta actividad se usa memoria separada: un puntero host no sustituye a un puntero device y los datos deben transferirse de forma explícita.

El índice global unidimensional se calcula mediante:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$$

Cada hilo procesa un índice independiente. Como el número de hilos lanzados puede superar el número de datos, todos los kernels incluyen la condición `i <N`.

2.1. Flujo general

1. Crear e inicializar datos en el host.
2. Reservar memoria device con `cudaMalloc`.
3. Copiar entradas mediante `cudaMemcpyHostToDevice`.
4. Calcular grid y block.
5. Lanzar el kernel.
6. comprobar el lanzamiento y sincronizar.
7. Copiar resultados mediante `cudaMemcpyDeviceToHost`.
8. Validar y liberar recursos.

3. Parte 1: suma de arreglos en GPU

3.1. Objetivo y datos

La primera parte suma dos arreglos de $N = 16$ elementos. La inicialización es:

$$A[i] = i + 1, \qquad B[i] = 2i + 3$$

Por lo tanto, el valor esperado es:

$$C[i] = A[i] + B[i] = 3i + 4$$

3.2. Kernel

```
__global__ void sumarArreglos(
    const int *A, const int *B, int *C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
```

```
    }
}
```

El programa usa bloques de ocho hilos y dos bloques. La condición de límites mantiene el kernel correcto incluso si se cambia a una configuración que lance más hilos que elementos.

3.3. Resultado observado

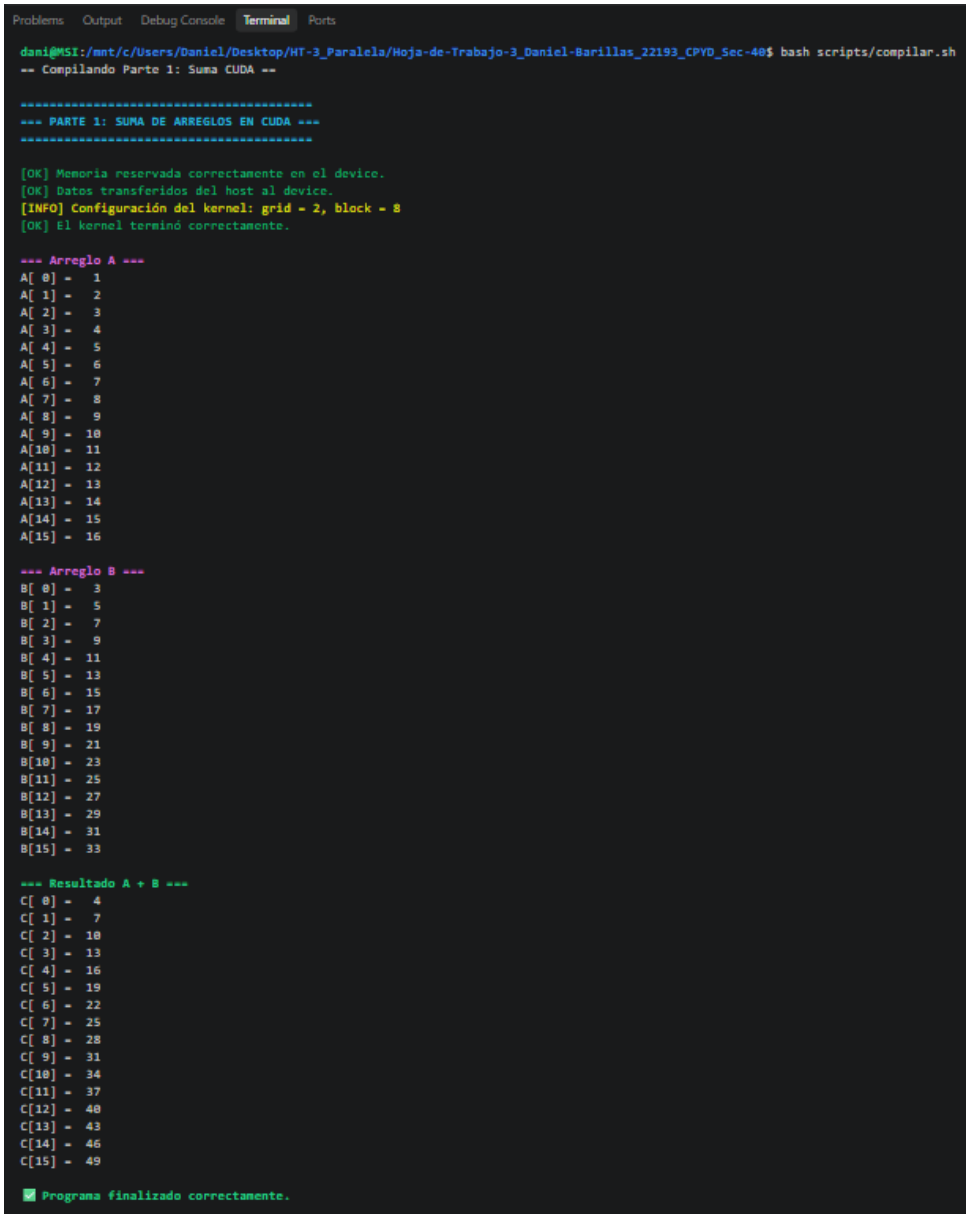


Figura 2: Compilación y ejecución de la suma de arreglos en CUDA.

La Figura 2 muestra $C[0] = 4$, $C[1] = 7$ y una progresión de tres unidades hasta $C[15] = 49$. Estos valores coinciden exactamente con $3i + 4$. La captura también evidencia la configuración `grid = 2` y `block = 8`, además del regreso de la salida al host.

3.4. Respuestas requeridas

3.4.1. ¿Qué partes se ejecutan en CPU y cuáles en GPU?

La CPU ejecuta `main`, inicializa A y B, invoca `cudaMalloc`, ordena las copias, calcula la configuración de lanzamiento, verifica errores, imprime la salida y libera memoria. La GPU ejecuta únicamente el cuerpo del kernel: cada hilo calcula un índice y suma dos enteros.

3.4.2. ¿Qué ocurre al cambiar grid y block?

El total de hilos es *gridSize* $\times$ *blockSize*. Si es menor que N, algunos elementos nunca se procesan. Si es mayor, la comprobación `idx < N` descarta los hilos sobrantes. El tamaño de bloque también afecta el rendimiento: bloques demasiado pequeños pueden desaprovechar recursos, mientras bloques muy grandes pueden reducir ocupación o exceder límites del dispositivo. Por ello se usa la división de techo:

$$\mathit{gridSize} = \left\lceil \frac{N}{\mathit{blockSize}} \right\rceil$$

3.4.3. ¿Qué ocurre si no se reserva o libera memoria?

Sin `cudaMalloc`, las direcciones device no son válidas y las copias o el kernel devuelven errores. Liberar memoria antes de terminar causa accesos inválidos. No liberarla al final produce una fuga que puede agotar la memoria de la GPU en ejecuciones repetidas.

3.4.4. ¿Qué ocurre si no se transfiere el resultado a CPU?

El arreglo calculado permanece en `d_C`. `h_C` no se actualiza de forma automática porque host y device usan espacios de memoria distintos. Sin `cudaMemcpyDeviceToHost`, la CPU no dispone de los valores para imprimirlos o utilizarlos en otra operación.

4. Parte 2: clasificación por umbral

4.1. Relación con la plantilla

La actividad solicita completar `vacio.cu`. El repositorio conserva dos versiones:

- `docs/material_original/vacio_original.cu`: plantilla intacta;
- `source/vacio.cu`: solución completada y entregable.

También se mantiene `parte2_umbral_clasificacion.cu`, una variante con mensajes de diagnóstico, y `umbralizar.cu`, cuyo vocabulario sigue de cerca al archivo original.

4.2. Regla de clasificación

Para $N = 32$, el arreglo contiene $0, 5, 10, \ldots, 155$. La salida se define:

$$\mathit{salida}[i] = \begin{cases} 1, & \mathit{entrada}[i] \geq 50 \\ 0, & \mathit{entrada}[i] < 50 \end{cases}$$

```
__global__ void clasificarUmbral(
    const int *entrada, int *salida, int N, int umbral) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        salida[idx] = (entrada[idx] >= umbral) ? 1 : 0;
    }
}
```

4.3. Resultados

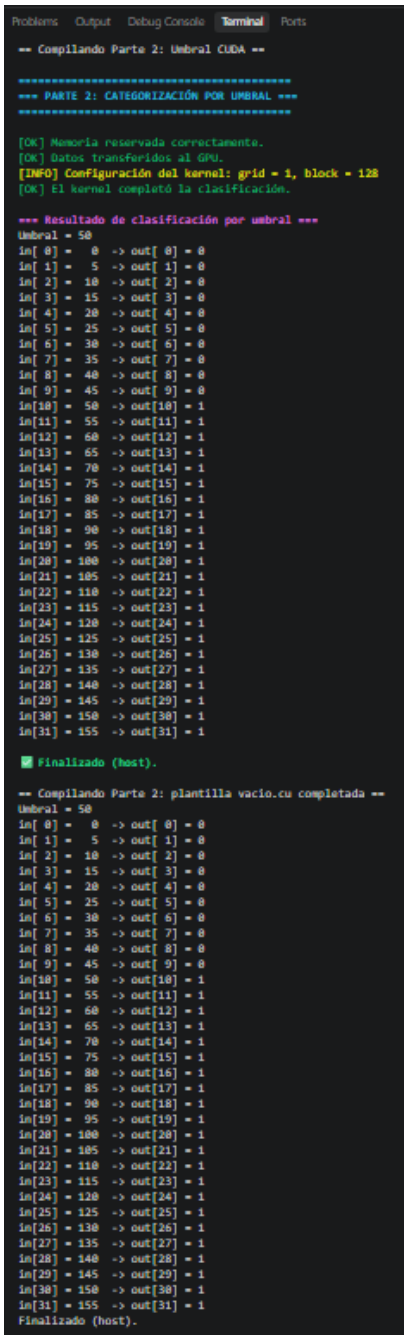


Figura 3: Ejecución de la clasificación por umbral con mensajes de diagnóstico.

La Figura 3 confirma que los índices 0 a 9, cuyos valores son menores que 50, producen cero. El índice 10 contiene 50 y produce uno; todos los índices restantes mantienen esa clasificación.

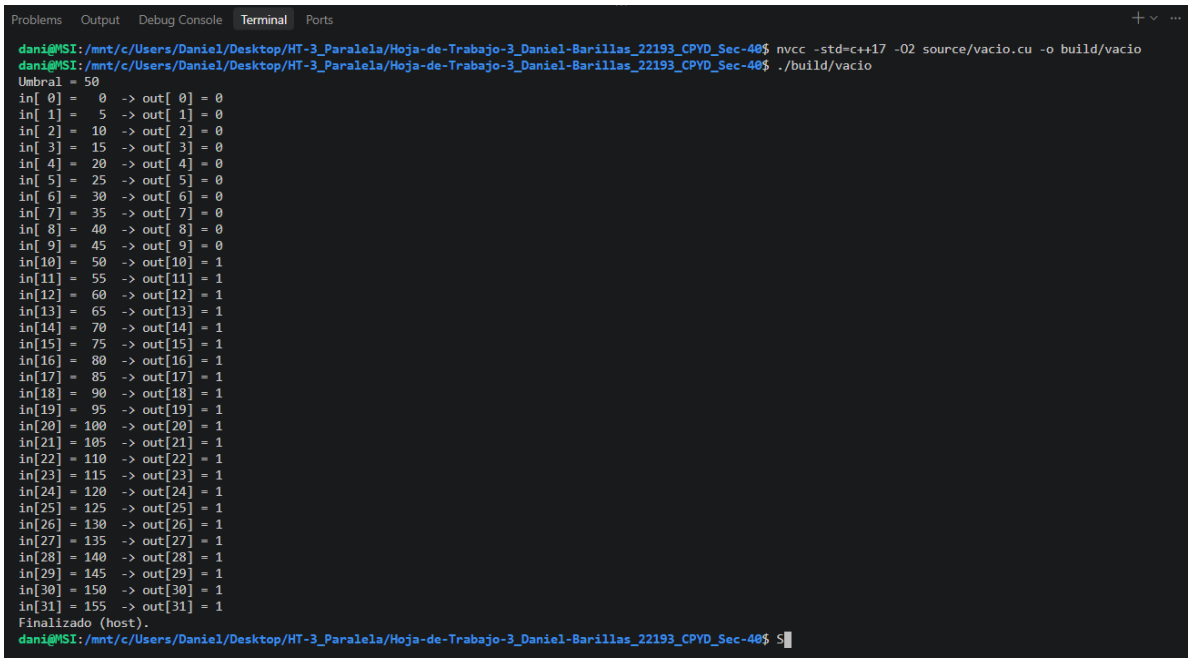


Figura 4: Ejecución de la plantilla `vacio.cu` después de completarla.

La Figura 4 valida específicamente el archivo solicitado por el enunciado. Ambas implementaciones producen la misma secuencia, lo que elimina la ambigüedad sobre cuál condición de umbral se usó.

5. Parte 3: versión CPU y comparación de tiempos

5.1. Versión exclusivamente CPU

El archivo `parte3_cpu_vs_gpu.cpp` implementa la misma suma mediante un ciclo secuencial. El vector de salida se calcula así:

```
void sumarArreglosCPU(
    const std::vector<int>& A,
    const std::vector<int>& B,
    std::vector<int>& C) {
    for (std::size_t i = 0; i < A.size(); ++i) {
        C[i] = A[i] + B[i];
    }
}
```

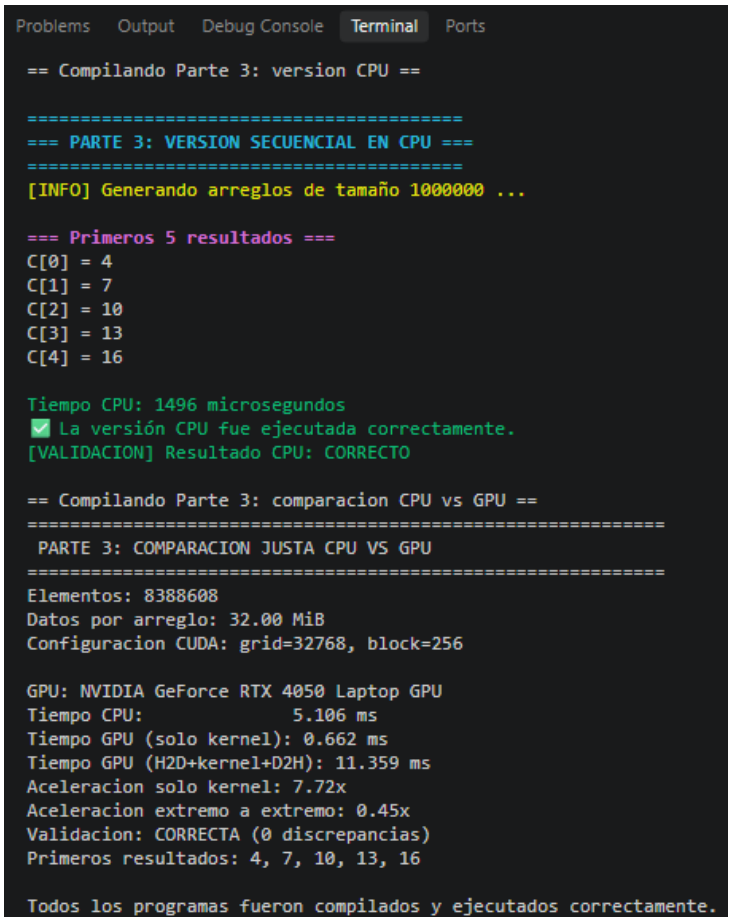


Figura 5: Ejecución de la versión CPU y de la comparación CPU/GPU.

La Figura 5 valida la versión secuencial para un millón de elementos, con 1 496 microsegundos, y muestra a continuación la comparación justa sobre 8 388 608 elementos. Los primeros valores $4, 7, 10, 13, 16$ coinciden en ambas implementaciones.

5.2. Diseño de la comparación justa

El nuevo archivo `parte3_comparacion_cpu_gpu.cu` ejecuta CPU y GPU sobre los mismos 8 388 608 elementos. Reserva los vectores de salida antes de medir, realiza un calentamiento CUDA y comprueba todos los resultados.

Se informan dos tiempos GPU:

1. **Solo kernel**: medido mediante `cudaEventRecord`; representa el cálculo en el dispositivo.
2. **Extremo a extremo**: incluye las copias H2D, el kernel y la copia D2H; excluye `cudaMalloc` y `cudaFree`.

Esta separación evita una conclusión engañosa. Una GPU puede ejecutar el kernel rápidamente y, aun así, perder ventaja si el problema es pequeño o las transferencias dominan el costo total.

5.3. Resultados medidos

Métrica	Resultado
Cantidad de elementos	8 388 608
Tiempo CPU	5.106 ms
Tiempo GPU, solo kernel	0.662 ms
Tiempo GPU, H2D + kernel + D2H	11.359 ms
Aceleración, solo kernel	7.72x
Aceleración extremo a extremo	0.45x
Discrepancias CPU/GPU	0

Cuadro 2: Resultados CPU/GPU obtenidos en la RTX 4050 Laptop GPU.

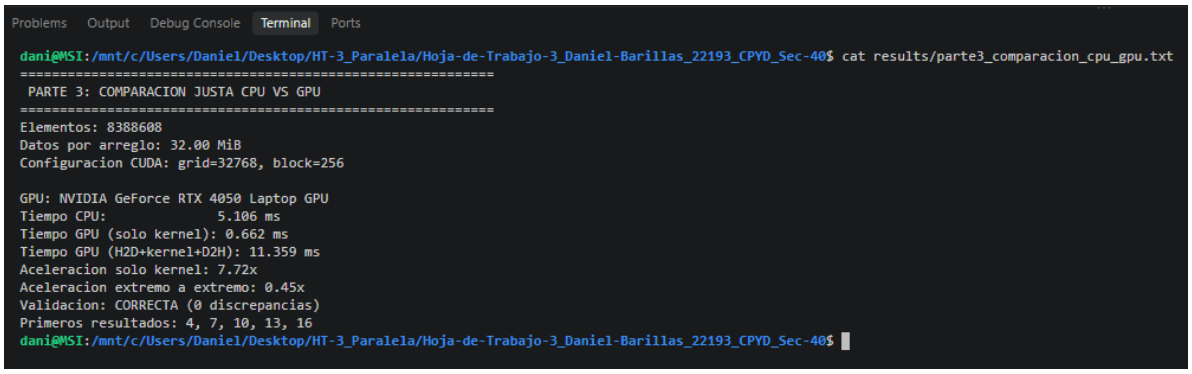


Figura 6: Salida completa del benchmark CPU/GPU y validación sin discrepancias.

El kernel tardó solo 0.662 ms frente a 5.106 ms de CPU, por lo que el cálculo paralelo aislado fue 7.72 veces más rápido. No obstante, las transferencias de 64 MiB de entrada y 32 MiB de salida elevaron el ciclo GPU a 11.359 ms. En este caso, la CPU completó la operación extremo a extremo en aproximadamente el 45 % del tiempo GPU. Esto no contradice la ventaja del kernel: demuestra que el costo de comunicación domina cuando los datos se transfieren para una sola operación sencilla.

6. Validación integral

Parte	Criterio	Resultado
Parte 1	$C[i] = 3i + 4$ para los 16 elementos	**Cumplido**
Parte 2	0 para valores menores que 50 y 1 desde 50	**Cumplido**
CPU	Primeros valores 4, 7, 10, 13 y 16; verificación completa	**Cumplido**
CPU/GPU	Mismos datos y cero discrepancias	**Cumplido**
Memoria	Reserva, copia, sincronización y liberación comprobadas	**Cumplido**

Cuadro 3: Resumen de validación funcional.

La revisión del código añadió comprobaciones explícitas al retorno de `cudaDeviceSynchronize`. Esto evita anunciar que un kernel terminó correctamente cuando el lanzamiento fue válido, pero ocurrió un error durante su ejecución.

7. Reproducibilidad y limitaciones

7.1. Reproducibilidad

El script `scripts/compilar.sh` compila y ejecuta todos los programas. Cada salida se conserva como texto en `results/`. Los ejecutables se guardan en `build/`, directorio excluido de la entrega para respetar la instrucción de no entregar binarios.

7.2. Factores que afectan el tiempo

Los tiempos dependen del procesador, modelo de GPU, versión del controlador, frecuencias, temperatura, procesos concurrentes y optimizaciones del compilador. Por ello:

- no se comparan capturas obtenidas con tamaños diferentes;
- no se trasladan tiempos desde otro equipo;
- se documenta qué operaciones incluye cada medición;
- se valida corrección antes de interpretar velocidad.

8. Conclusiones

La suma vectorial demuestra un caso de paralelismo de datos directo: cada hilo trabaja sobre un elemento y no necesita comunicarse con otros hilos. Los resultados $4, 7, 10, \ldots, 49$ confirman que la indexación, las transferencias y el kernel son correctos.

La clasificación por umbral consolida el mismo patrón con una decisión independiente por dato. La transición exacta en 50 valida la condición `entrada[idx] >= umbral` y demuestra que la plantilla original fue completada de acuerdo con el enunciado.

La Parte 3 distingue correctamente entre demostrar paralelismo y demostrar rendimiento. El kernel GPU alcanzó una aceleración de 7.72x, pero la ejecución extremo a extremo obtuvo 0.45x al incluir transferencias. La GPU fue más rápida en cómputo y más lenta en el ciclo completo, lo que evidencia el peso de la comunicación para una operación aritmética sencilla.

En conjunto, la actividad evidencia el ciclo completo de una aplicación CUDA: preparación en CPU, administración explícita de memoria, distribución en hilos, sincronización, retorno de datos y validación. Las tres partes fueron compiladas y ejecutadas, y la comparación final terminó con cero discrepancias.

A. Archivos principales

Archivo	Propósito
`source/parte1_suma_cuda.cu`	Suma paralela y respuestas de Parte 1
`source/vacio.cu`	Plantilla de Parte 2 completada
`source/parte2_umbral_clasificacion.cu`	Variante documentada del umbral
`source/parte3_cpu_vs_gpu.cpp`	Versión exclusivamente CPU
`source/parte3_comparacion_cpu_gpu.cu`	Benchmark CPU/GPU justo
`scripts/verificar_entorno.sh`	Verificación de compiladores y GPU
`scripts/compilar.sh`	Compilación, ejecución y captura de resultados